# SI 201 - Final Project Report

Team A&E // Emily Lask + Armana Akhsambiyeva

Our GitHub Repo:

https://github.com/elask-umich/si201-final-project.git

The report must include:

1. The goals for your project including what APIs/websites you planned to work with and what data you planned to gather (10 points)

For this Final Project, Armana and I had a goal to relate and identify the cross section between the list of characters from the Harry Potter movies and a related Youtube channel. This pop-culture theme became extremely relevant on Youtube as it is one of the longest standing media sharing platforms on the internet and it cultivates many community circles for large fandoms. We focused on the favoritism of main characters and how they show up on the selected channel by displaying popularity and interests within the fandom via views and mentions on a collection of videos. To satisfy our goal, we decided to gather the data of all youtube videos from this channel, the video stats (durations, view count, duration, video title, etc.) and all of the listed information about each character from the Harry Potter API (name, house, gender, birthyear, role, species, etc.)

*These are the APIs we included in our work:*

Harry Potter API - https://hp-api.onrender.com/api/characters

Youtube APIs - https://github.com/youtube/api-samples

Youtube Channel Harry Potter Theory

2. The goals that were achieved including what APIs/websites you actually worked with and what data you did gather (10 points)

We successfully reached our ultimate goal of finding out how much this theme/topic appeared on Youtube by narrowing down our scope to the single Youtube channel. We actually expanded on the amount of data we gathered to accurately calculate our data for our resulting visualizations. We created tables for inclusive data like character name mentions per video title/video id, using references, and even the like count per video, comment count, a video's view to like ratio, total view counts, and the video's publishing date/time. We did not differ or expand beyond the APIs and the Youtube Channel listed above.

3. The problems that you faced (10 points)

We faced two main challenges when starting and working on the project. The first being that the initial API for Harry Potter we used (the one listed on the github repository with all potential APIs that was given to us by the SI201 team) did not contain information that was up to date (it was from a few years ago), and did not contain enough information required for each of the characters. This is the reason why Emily and I found an entirely different open source API to use for the Harry Potter API. The second problem was regarding the visuals. When I (Armana) was coding the pie chart meant for the second. We also ran into some problems when creating the 'character_mentions' table, as somewhere along the way it had broken our imports for the youtube API data, so I (Emily) had to debug and ensure all naming conventions were correct for the data to collect from the source database. Somehow the naming conventions for title were the same for the channel title and youtube video title, so when I (Emily) was running the database schema, it was overriding the video titles as the channel titles, and that impacted the visualizations and data for calculations to be skewed.

Another major issue that we are currently facing is the data skewness after removal of the duplicate string data from the harry potter db (character table) and the combined db update after fixing the harry potter tables. Our current visualizations are accurate, despite them looking off, because they actually reflect

4. The calculations from the data in the database (i.e. a screen shot) (10 points)

```python
def calc_character_popularity(db):
    conn = sqlite3.connect(db)
    cur = conn.cursor()

    cur.execute("SELECT id, name FROM characters")
    chars = cur.fetchall()
    results = {}

    for cid, name in chars:
        cur.execute("""
            SELECT v.id FROM videos v
            WHERE lower(v.title) LIKE ?
        """, (f"%{name.lower()}%",))
        vids = cur.fetchall()

        views = 0
        for (vid,) in vids:
            cur.execute("""
                SELECT view_count FROM video_stats
                WHERE video_ref=?
            """, (vid,))
            r = cur.fetchone()
            if r and r[0]:
                views += r[0]

        results[name] = {"mentions": len(vids), "views": views}

    conn.close()
    return results
```

```python
def build_char_mentions(db):
    conn = sqlite3.connect(db)
    cur = conn.cursor()

    cur.execute("SELECT id, name FROM characters")
    chars = cur.fetchall()
    cur.execute("SELECT id, title FROM videos")
    vids = cur.fetchall()

    added = 0
    for cid, name in chars:
        if not name:
            continue
        for vid, title in vids:
            if title and name.lower() in title.lower():
                cur.execute("""
                    SELECT 1 FROM character_mentions
                    WHERE character_ref=? AND video_id=?
                """, (cid, vid))
                if cur.fetchone():
                    continue
                cur.execute("""
                    INSERT INTO character_mentions
                    VALUES (NULL, ?, ?, 1)
                """, (cid, vid))
                added += 1

    conn.commit()
    conn.close()
    print(f"✓ character_mentions updated ({added} rows)")
```

```python
def export_calculations(db, out="hp_stats.txt"):
    stats = calc_character_popularity(db)
    with open(out, "w", encoding="utf-8") as f:
        for name, s in stats.items():
            f.write(f"{name}\n")
            f.write(f"  Mentions: {s['mentions']}\n")
            f.write(f"  Views: {s['views']}\n\n")
    print(f"✓ {out} written")
```

```python
# ------------- calculations for both (placeholder start) ------------------

def calc_character_popularity(final_db_path: str):
    """
    Counts how many YouTube videos mention each Harry Potter character in the title,
    and sums the view_count for those videos.
    """
    conn = sqlite3.connect(final_db_path)
    cur = conn.cursor()

    # Get all HP characters
to add a breakpoint 'SELECT id, name FROM characters")
    characters = cur.fetchall()

    results = {}

    for char_id, name in characters:
        if not name:
            continue

        name_lower = f"%{name.lower()}%"

        # Find all video IDs where the title contains this character's name
        cur.execute("""
            SELECT v.id
            FROM videos v
            WHERE lower(v.title) LIKE ?
        """, (name_lower,))
        matching_videos = cur.fetchall()

        # Count how many titles mention the character
        mention_count = len(matching_videos)

        # Sum up total views from video_stats
        total_views = 0
        for (vid_id,) in matching_videos:
```

```python
        total_views = 0
        for (vid_id,) in matching_videos:
            cur.execute("""
                SELECT view_count
                FROM video_stats
                WHERE video_ref = ?
            """, (vid_id,))
            stat = cur.fetchone()
            if stat and stat[0]:
                total_views += stat[0]

        results[name] = {
            "mentions": mention_count,
            "views": total_views
        }

    conn.close()
    return results


def calc_character_appearances_in_yt_videotitle(final_db_path: str):
    conn = sqlite3.connect(final_db_path)
    cur = conn.cursor()

    cur.execute("SELECT id, name FROM characters")
    characters = cur.fetchall()

    cur.execute("SELECT id, title FROM videos")
    videos = cur.fetchall()

    conn.close()

    results = []

    for char_id, name in characters:
        if not name:
            continue
```

```python
def calc_character_appearances_in_yt_videotitle(final_db_path: str):
    conn.close()

Click to collapse the range.

    for char_id, name in characters:
        if not name:
            continue

        name_lower = name.lower()
        count = sum(
            1 for _, title in videos
            if title and name_lower in title.lower()
        )

        results.append((name, count))

    results.sort(key=lambda x: x[1], reverse=True)
    return results


# -------------------- Return Calc to TXT files --------------------

def export_calculations_to_txt(db_path="combined.db", output_file="hp_stats.txt"):
    conn = sqlite3.connect(db_path)
    stats = calc_character_popularity(db_path)

    with open(output_file, "w", encoding="utf-8") as f:
        for name, info in stats.items():
            f.write(f"{name}\n")
            f.write(f"  Mentions in video titles: {info['mentions']}\n")
            f.write(f"  Total views of those videos: {info['views']}\n\n")

    conn.close()
    print(f"TXT generated: {output_file}")
```

hp_stats.txt

```
Harry Potter
  Mentions in video titles: 143
  Total views of those videos: 19409838

Hermione Granger
  Mentions in video titles: 0
  Total views of those videos: 0

Ron Weasley
  Mentions in video titles: 0
  Total views of those videos: 0

Draco Malfoy
  Mentions in video titles: 1
  Total views of those videos: 59290

Minerva McGonagall
  Mentions in video titles: 0
  Total views of those videos: 0

Cedric Diggory
  Mentions in video titles: 1
  Total views of those videos: 56919

Cho Chang
  Mentions in video titles: 0
  Total views of those videos: 0

Severus Snape
  Mentions in video titles: 1
  Total views of those videos: 107907

Rubeus Hagrid
  Mentions in video titles: 1
  Total views of those videos: 126554

Neville Longbottom
  Mentions in video titles: 1
  Total views of those videos: 19186
```

hp_stats.txt

```
Luna Lovegood
    Mentions in video titles: 0
    Total views of those videos: 0

Ginny Weasley
    Mentions in video titles: 0
    Total views of those videos: 0

Sirius Black
    Mentions in video titles: 0
    Total views of those videos: 0

Remus Lupin
    Mentions in video titles: 0
    Total views of those videos: 0

Arthur Weasley
    Mentions in video titles: 0
    Total views of those videos: 0

Bellatrix Lestrange
    Mentions in video titles: 1
    Total views of those videos: 107826

Lord Voldemort
    Mentions in video titles: 0
    Total views of those videos: 0

Horace Slughorn
    Mentions in video titles: 1
    Total views of those videos: 102027

Kingsley Shacklebolt
    Mentions in video titles: 0
    Total views of those videos: 0

Dolores Umbridge
    Mentions in video titles: 0
    Total views of those videos: 0
```

hp_stats.txt

```
Lucius Malfoy
    Mentions in video titles: 0
    Total views of those videos: 0

Vincent Crabbe
    Mentions in video titles: 0
    Total views of those videos: 0

Mr Crabbe
    Mentions in video titles: 0
    Total views of those videos: 0

Gregory Goyle
    Mentions in video titles: 0
    Total views of those videos: 0

Mr Goyle
    Mentions in video titles: 0
    Total views of those videos: 0

Mrs Norris
    Mentions in video titles: 0
    Total views of those videos: 0

Argus Filch
    Mentions in video titles: 0
    Total views of those videos: 0

Vernon Dursley
    Mentions in video titles: 0
    Total views of those videos: 0

Petunia Dursley
    Mentions in video titles: 0
    Total views of those videos: 0

Dudley Dursley
    Mentions in video titles: 0
    Total views of those videos: 0
```

hp_stats.txt

```
Gellert Grindelwald
  Mentions in video titles: 0
  Total views of those videos: 0

Norberta
  Mentions in video titles: 0
  Total views of those videos: 0

Ronan
  Mentions in video titles: 0
  Total views of those videos: 0

Bane
  Mentions in video titles: 0
  Total views of those videos: 0

Firenze
  Mentions in video titles: 0
  Total views of those videos: 0

The Giant Squid
  Mentions in video titles: 0
  Total views of those videos: 0

Murcus
  Mentions in video titles: 0
  Total views of those videos: 0

Elfrick the Eager
  Mentions in video titles: 0
  Total views of those videos: 0

Perenelle Flamel
  Mentions in video titles: 0
  Total views of those videos: 0
```

character_appearances.txt

```
=== Character Appearances in YouTube Video Titles ===

Harry Potter: 123 video title matches
Tom: 2 video title matches
Draco Malfoy: 1 video title matches
Severus Snape: 1 video title matches
Rubeus Hagrid: 1 video title matches
Neville Longbottom: 1 video title matches
Bellatrix Lestrange: 1 video title matches
Horace Slughorn: 1 video title matches
James Potter: 1 video title matches
Newt Scamander: 1 video title matches
Hermione Granger: 0 video title matches
Ron Weasley: 0 video title matches
Minerva McGonagall: 0 video title matches
Cedric Diggory: 0 video title matches
Cho Chang: 0 video title matches
Luna Lovegood: 0 video title matches
Ginny Weasley: 0 video title matches
Sirius Black: 0 video title matches
Remus Lupin: 0 video title matches
Arthur Weasley: 0 video title matches
Lord Voldemort: 0 video title matches
Kingsley Shacklebolt: 0 video title matches
Dolores Umbridge: 0 video title matches
Lucius Malfoy: 0 video title matches
Vincent Crabbe: 0 video title matches
Mr Crabbe: 0 video title matches
Gregory Goyle: 0 video title matches
Mr Goyle: 0 video title matches
Mrs Norris: 0 video title matches
Argus Filch: 0 video title matches
Vernon Dursley: 0 video title matches
Petunia Dursley: 0 video title matches
Dudley Dursley: 0 video title matches
Boa constrictor: 0 video title matches
Lily Potter: 0 video title matches
Albus Dumbledore: 0 video title matches
Fawkes: 0 video title matches
```

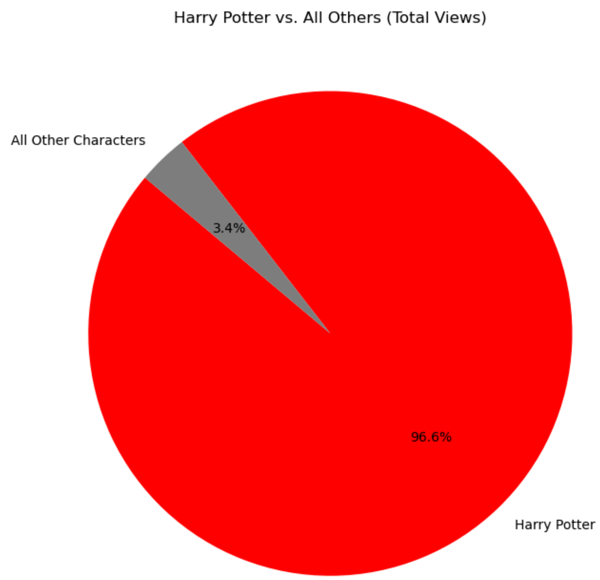
character_appearances.txt

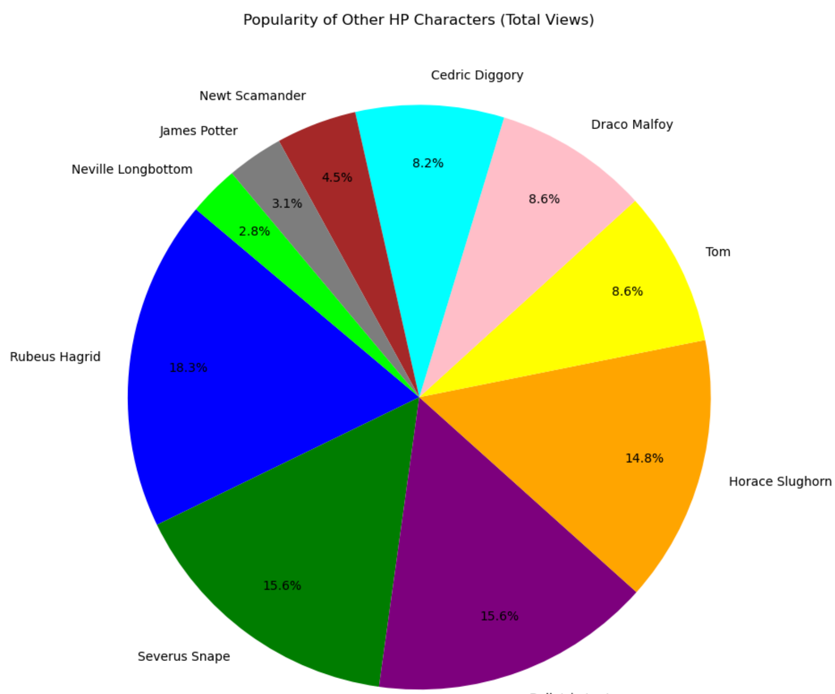
```
Madam Pomfrey: 0 video title matches
Mrs Figg: 0 video title matches
Marge Dursley: 0 video title matches
Ripper: 0 video title matches
Yvonne: 0 video title matches
Piers Polkiss: 0 video title matches
Dennis: 0 video title matches
Malcolm: 0 video title matches
Gordon: 0 video title matches
Miranda Gaushawk: 0 video title matches
Bathilda Bagshot: 0 video title matches
Adalbert Waffling: 0 video title matches
Emeric Switch: 0 video title matches
Phyllida Spore: 0 video title matches
Arsenius Jigger: 0 video title matches
Quentin Trimble: 0 video title matches
Doris Crockford: 0 video title matches
Quirinus Quirrel: 0 video title matches
Griphook: 0 video title matches
Madam Malkin: 0 video title matches
Vindictus Viridian: 0 video title matches
Garrick Ollivander: 0 video title matches
Hedwig: 0 video title matches
Trevor: 0 video title matches
Hermes: 0 video title matches
Errol: 0 video title matches
Pigwidgeon: 0 video title matches
Molly Weasley: 0 video title matches
Percy Weasley: 0 video title matches
Fred Weasley: 0 video title matches
George Weasley: 0 video title matches
Lee Jordan: 0 video title matches
Bill Weasley: 0 video title matches
Charlie Weasley: 0 video title matches
Fat Friar: 0 video title matches
Peeves: 0 video title matches
Hannah Abbott: 0 video title matches
Susan Bones: 0 video title matches
Terry Boot: 0 video title matches
```

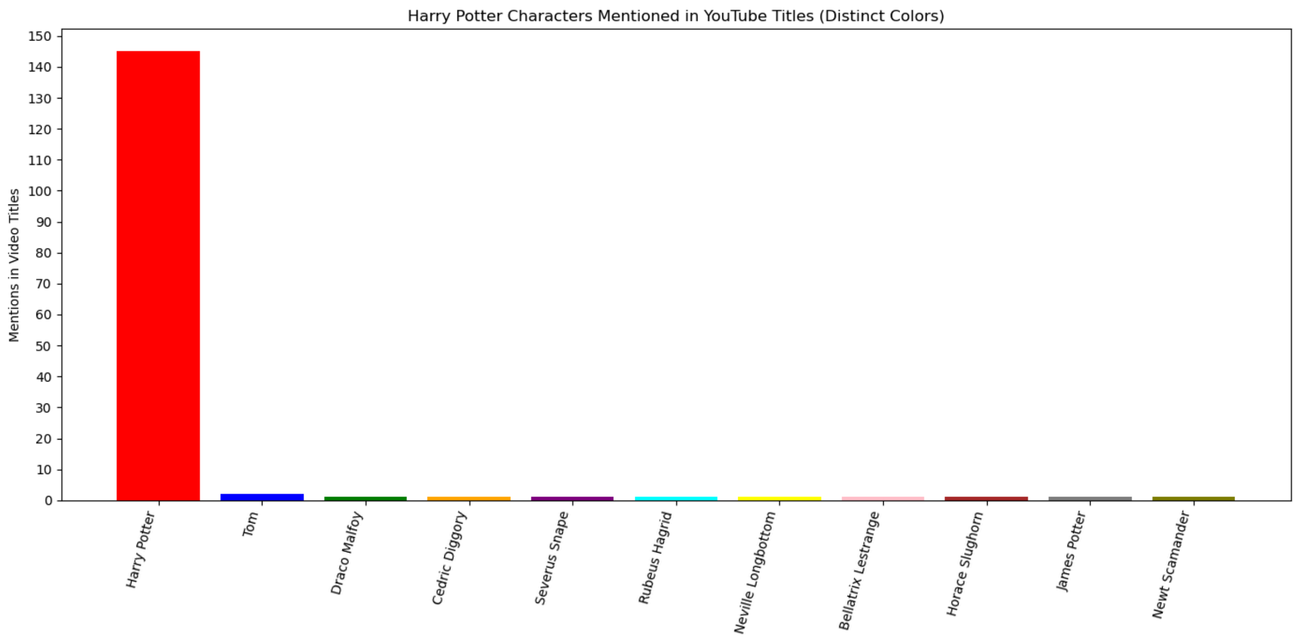
character_appearances.txt

```
Pigwidgeon: 0 video title matches
Molly Weasley: 0 video title matches
Percy Weasley: 0 video title matches
Fred Weasley: 0 video title matches
George Weasley: 0 video title matches
Lee Jordan: 0 video title matches
Bill Weasley: 0 video title matches
Charlie Weasley: 0 video title matches
Fat Friar: 0 video title matches
Peeves: 0 video title matches
Hannah Abbott: 0 video title matches
Susan Bones: 0 video title matches
Terry Boot: 0 video title matches
Mandy Brocklehurst: 0 video title matches
Lavender Brown: 0 video title matches
Millicent Bulstrode: 0 video title matches
Justin Finch-Fletchley: 0 video title matches
Seamus Finnigan: 0 video title matches
Mrs Finnigan: 0 video title matches
Mr Finnigan: 0 video title matches
Morag MacDougal: 0 video title matches
Lily Moon: 0 video title matches
Theodore Nott: 0 video title matches
Mr Nott: 0 video title matches
Pansy Parkinson: 0 video title matches
Parvati Patil: 0 video title matches
Padma Patil: 0 video title matches
Sally-Anne Perks: 0 video title matches
Lisa Turpin: 0 video title matches
Blaise Zabini: 0 video title matches
Nearly Headless Nick: 0 video title matches
Bloody Baron: 0 video title matches
The Wailing Widow: 0 video title matches
Fat Lady: 0 video title matches
Pomona Sprout: 0 video title matches
Cuthbert Binns: 0 video title matches
Emeric the Evil: 0 video title matches
```

5. The visualization that you created (i.e. screen shot or image file) (10 points)


Harry Potter vs. All Others (Total Views)
All Other Characters
3.4%
96.6%
Harry Potter


Popularity of Other HP Characters (Total Views)
Cedric Diggory
8.2%
Newt Scamander
4.5%
Draco Malfoy
8.6%
James Potter
3.1%
Neville Longbottom
2.8%
Tom
8.6%
Rubeus Hagrid
18.3%
Horace Slughorn
14.8%
Severus Snape
15.6%
Bellatrix Lestrange
15.6%




6. Instructions for running your code (10 points)
   The instructions are pasted from the github repo;

   Hello! This is Team A&E's Final project and all files, please read these instructions for running our final python file.

   You will open a bash terminal and run the command listed below. DO SO ONLY IF you need to
   1. Generate the hp_stats.txt 2. Generate the character_appearances.txt. and/or need to run and add lines to 3. the final combined.db database.type - python harryportter_youtube_db.py --youtube-src youtube_db.db --import-hp hp_db.db --limit 25.

   To see our visualizations, please open the visualization.py file and run it as is

7. An updated function diagram with the names of each function, the input, and output and who was responsible for that function (20 points)

Armana - purple functions and the files listed below:

1. Harrypotter_fetch.py – all functions within this file are written by Armana Akhsambiyeva
    a. Init_db
    b. Get_hp_char
    c. Get_or_create_id
    d. Gather_store_hp
2. Harrypotter_youtube_db.py (partially) – functions Armana was responsible for:
    a. create_final_schema (added 2 cur.execute and debugged)
    b. import_youtube_from_source (debugged to update the column name and resolve major bug)
    c. import_hp
    d. calc_character_popularity
    e. copy_lookup_table
    f. Final revisions of all functions in the file, debugging of all functions associated.

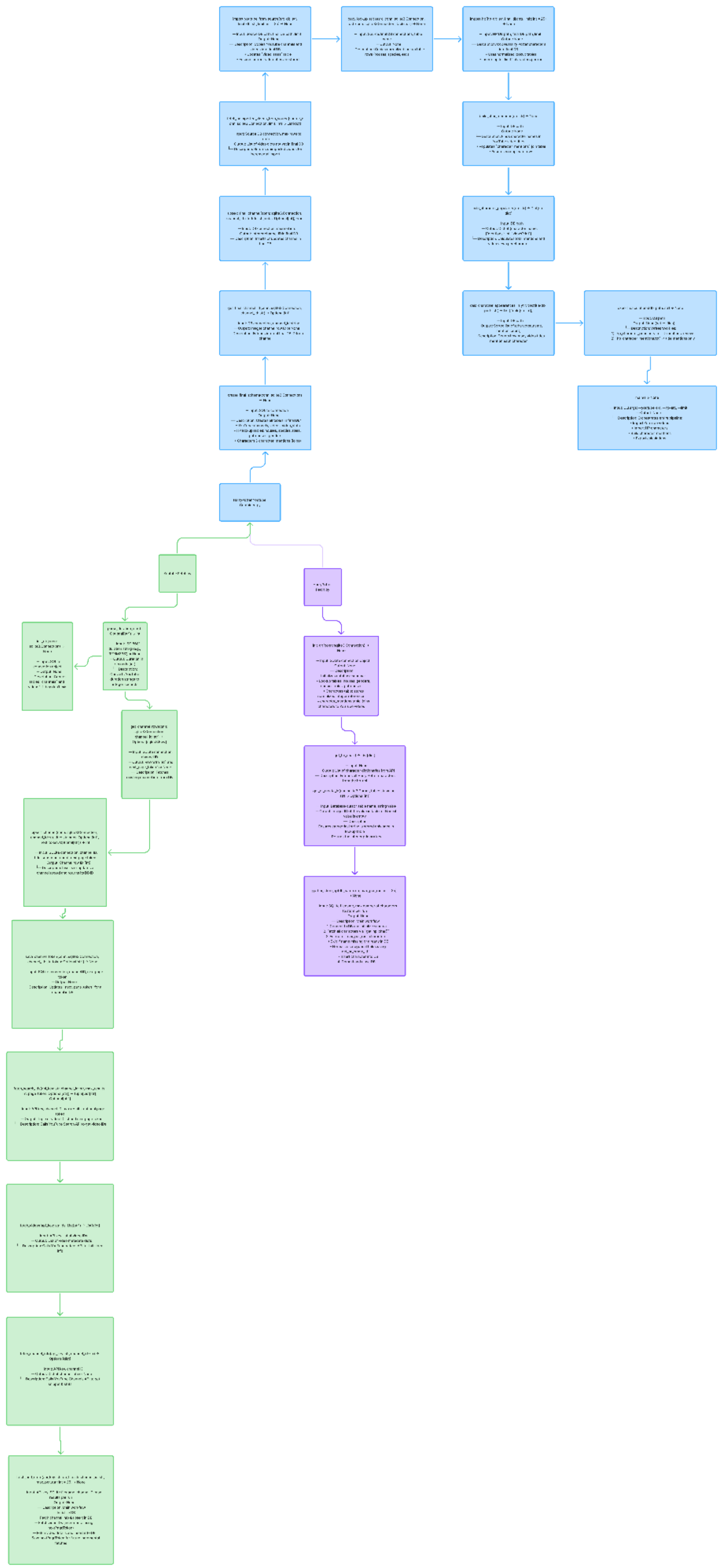
3. Hp_db.db (from harrypotter_fetch.py)
4. Visualization.py – all functions within this file are written by Armana Akhsambiyeva
    a. pie_harry_vs_rest
    b. pie_other_characters
    c. plot_character_title_mentions_bar
5. combined.db (partially, based on harrypotter_youtube_db.py)

Emily - green functions and the files listed below:

1. Youtube_fetch.py
    a. Wrote all functions in this .py and created the youtube_db.db
    b. Parse_duration_iso
    c. Init_db
    d. Get_channel_row
    e. Upsert_channel
    f. Save_channel_token
    g. Fetch_search_ids
    h. Fetch_videos
    i. Fetch_channel_info
    j. Fetch_and_store
2. harrypotter_youtube_db.py (partially) – functions Emily was responsible for:
    a. create_final_schema (wrote the bulk of and debugged)
    b. Get_final_channel

c. Upsert_final_channel
d. Fetch_unimported_videos_from_source
e. Import_youtube_from_source (wrote the bulk of)
f. Calc_character_appearances_in_yt_videotitle
g. Export_calculations
h. Main ()

3. Youtube_db.db
    a. This is the file created for the youtube specific API data
4. Instructions.txt
    a. Wrote a short instructional sheet for the full zip file containing our project
5. combined.db (partially)
    a. Input the data and source content for the youtube data in tables: channels, videos, and video_stats



8. You must also clearly document all resources you used. The documentation should be labelled using the table below (20 points)

| **Date:** | **Issue Description:** | **Location of Resource:** | **Result of Resource:** |
|---|---|---|---|
| (Armana)November 25th 2025 | (Armana)Mapped out and cleaned the fetch file functions for Harry Potter - meaning I wrote the functions (gather store and init) after prompting chatgpt to tell me the outline (without code) and wrote the raw function (without edits). Afterwhich I put it into chatgpt to edit and clean, to look cohesive and fix any potential logical or small misspelling errors. | (Armana)Harrypotter_fetch.py | (Armana) attributed to 3 commits from Armana on Nov. 25. Fixed the code, it ran smoothly and created the harry potter db (still with duplicate string data that was unaccounted for until after December 8th). This is labeled as "third commit, storing 25 characters per run and columns" and "second function complete, requests character information from the api and has a just in case feature in case the program is" and "fourth commit, debugging of the table create function, missing column name for alternate name and parenthesis bug fix" |

| | | | |
|---|---|---|---|
| (Armana) November 30th, 2025 | (Armana) There was no particular issue, but i wanted to use chatgpt to map out the logical flow of the functions i was writing for the combined database schema, as well as the calculations, I ended up debugging the functions with chatgpt as well. | (Armana) CHATGPT harrypotter_youtube_db.py | attributed to 3 commits from Armana on Nov. 30. Working functions and debugged spelling errors in the import_hp_placeholder function and the first calculation function calc_character_popularity |

| | | | |
|---|---|---|---|
| (Emily) November 25th 2025 | I ran into issues when initially writing and adding files into the repo on GitHub and had to delete my first draft of the youtube fetch file and the youtube_db and repush them to the repo after utilizing source control as an extension in vs code | (Emily) CHATGPT youtube_fetch.py Youtube_db.db VSCode | VS Code Repo extension working successfully so that the source files for my API imported into the combined database schema for combined_db.db |
| (Armana) December 8th 2025 | I was having issues with the visualizations, because the graphics and labels displayed were either too close together and illegible, or too far away from the pie chart and bar chart itself (for the bar chart, the issue was more related to the the y-axis labels, or the number step count for youtube mentions in titles being way too close together) | CHATGPT visualizations.py | The end result ended up being three separate visualizations, two pie charts for the second function that counted popularity of harry potter characters by the aggregate view count → this development was of my own thinking and not chatgpt, but chatgpt did end up debugging the second pie chart that only focused on characters outside of harry potter himself to measure popularity of characters in an unskewed way in at least one chart. The second calculation had the bar chart associated with it, and the y-axis was simply fixed by resizing the step count to count by 10s, when we initially did larger step counts. – chatgpt helped add the appropriate step count, the colors for the bar chart and pie |

| | | | |
|---|---|---|---|
| | | | charts, and create the mapping for the two separate pie charts. |
| (Emily) December 8th 2025 | During this session I worked on the building the final db and reworked/debugged three functs for build char mentions, export to txt, and second calc, updated main<br>I used chatgpt to instruct and help prompt how to structure the order of each functions | ChatGPT<br>Youtube_harrypotter_db.py<br>combined_db.db | I was able to take away the information on debugging and restructuring from chat and to finally organize and get the main funct to perform and the calculations to export properly |
| (Armana) December 15th | Used chatgpt to debug the main function and reimplement the second calculation that ended up being accidentally deleted in the process of redoing the combined db when removing the duplicate string data from harry potter db and the youtube videos table from the youtube db in the combined db. I used chatgpt to look over the old second calculation written before December 8th, and the new export, and had it look over what needed to be changed in the old function to work with the new. Nothing major needed changing but I did need to update the main to include the export function to the | CHATGPT<br>Harrypotter_youtube_db.db | The final product ended up resulting in two separate textfiles for the character mentioned by video title and the character popularity based on views. The functions work perfectly fine in tandem with the visualizations and the visualizations reflect the statistics shown in the textfiles.<br>Created separate tables with gender, patronus, house and roles, no more string duplication to my knowledge. |

| | | | |
|---|---|---|---|
| | text files. Also used chatgpt to create get_or_create_id function to ensure that duplicate string files, texts are only stored once and returns integer id → for patronus, house, role, and genders | | |
| (Emily + Armana) December 15th | After the presentation notes and reviewing the duplicate strings, Armana and I had to attempted to debug the video title that had incorrectly been labeled as channel title in the cur.execute in the import youtube from source → Armana ended up fixing it using chatgpt to debug (Armana here → I pasted the functions in isolation to chatgpt and looked over what may be the possible issue or where the mislabelling for the combined had occurred, because upon manual review, the original youtube videos table from youtube fetch in the youtube_db.db was correct and functioning properly, the combined db was faulty itself. I isolated it to the import youtube from source and fixed the issue by relabeling it to video titles.) | CHATGPT Harrypotter_youtube_db.py | The function ended up working perfectly, and the visualizations that were faulty also ended up being fixed in the process, the code now generates the correct video titles in the video table under combined, and the visualization code worked fine after the function was fixed, it inputted the correct characters outside of just harry potter which reflected on the pie charts and the bar charts. |